

Programmation en C

Les pointeurs, les structures et les fonctions

John Samuel

CPE Lyon

Année: 2025-2026

Courriel: john(dot)samuel(at)cpe(dot)fr



THE C PROGRAMMING LANGUAGE

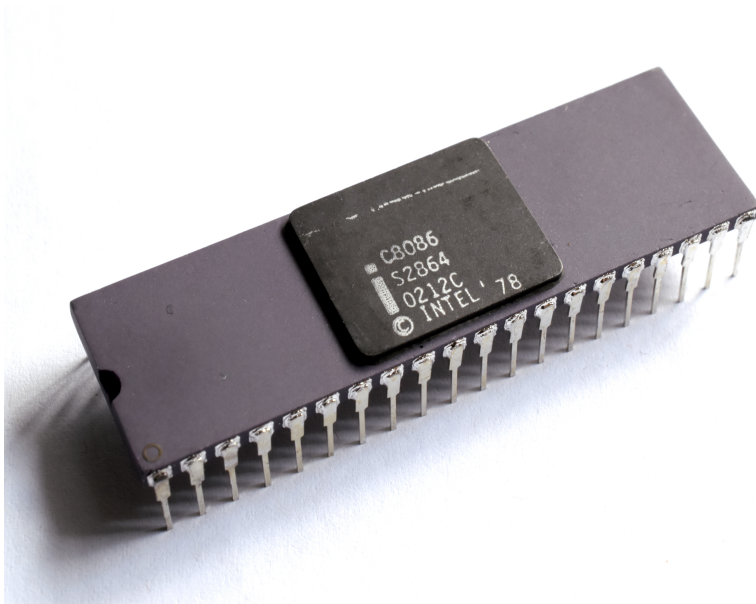
Objectifs

- Les pointeurs
- Les structures
- Les unions
- Introduction aux fonctions

2.2. Microprocesseur

Architecture

L'architecture d'une machine influence la programmation en C en déterminant les détails de la **gestion de la mémoire**, la **taille des types de données**, l'**ordre d'exécution des instructions** (gestion des threads sur un processeur multicœur), et d'autres **caractéristiques matérielles** (par exemple, la taille des registres), ce qui peut nécessiter des ajustements spécifiques lors de la rédaction de code pour différentes plateformes matérielles.



Architecture

- 16 bits: E.g., Intel C8086
- 32 bits: E.g., Intel 80386, Intel Pentium II/III, AMD Athlon
- 64 bits: E.g., AMD Athlon 64, Intel Core 2

Une architecture n-bit peut adresser une mémoire de taille 2^n .

2.2. Microprocesseur

Le nombre de bits d'une machine est étroitement lié à la mémoire de plusieurs manières :

- **Taille des données en mémoire** : Le nombre de bits détermine la quantité de données que la machine peut manipuler en une seule opération. Par exemple, sur une architecture 32 bits, les entiers sont généralement stockés sur 4 octets (32 bits), tandis que sur une architecture 64 bits, ils sont stockés sur 8 octets (64 bits). Cela influe sur la capacité de stockage et la manipulation des données en mémoire.
- **Adressage mémoire** : Le nombre de bits dans une adresse mémoire détermine la taille de l'espace d'adressage de la machine. Sur une architecture 32 bits, vous avez théoriquement un espace d'adressage de 2^{32} octets (4 Go), tandis qu'une architecture 64 bits offre un espace d'adressage de 2^{64} octets (plusieurs exaoctets). Cela affecte la capacité de la machine à adresser et à accéder à la mémoire.

2.2. Microprocesseur

Le nombre de bits d'une machine est étroitement lié à la mémoire de plusieurs manières :

- **Performance** : Les architectures 64 bits permettent souvent de manipuler de plus grandes quantités de données en une seule opération, ce qui peut améliorer les performances dans certaines situations. Cependant, cela peut également entraîner une utilisation de la mémoire plus intensive, ce qui nécessite de prendre en compte la gestion de la mémoire.
- **Compatibilité** : Les programmes écrits pour une architecture donnée (32 bits ou 64 bits) peuvent ne pas être compatibles avec une autre. La taille des pointeurs et des données affecte la manière dont le code interagit avec la mémoire, ce qui peut nécessiter des adaptations lors de la migration d'une architecture à l'autre.

2.2. Microprocesseur

Le nombre de bits dans une architecture détermine sa capacité à adresser et à gérer la mémoire, ce qui est essentiel pour la performance et la capacité des systèmes informatiques.

- 16 bits: 65,536 adresses (octets)
- 32 bits: 4 Gio (2^{32} :4,294,967,295) adresses (octets)
- 64 bits: 16 Eio (2^{64}) adresses (octets)

0	0	0	0	1	0	0	0
1	0	0	0	1	1	0	0
2	0	0	1	0	1	0	1
3	0	0	1	1	1	0	1
...							
...							
...							
...							
2^n-4	0	1	0	0	1	1	0
2^n-3	0	1	0	1	1	1	0
2^n-2	0	1	1	0	1	1	1
2^n-1	0	1	1	1	1	1	1

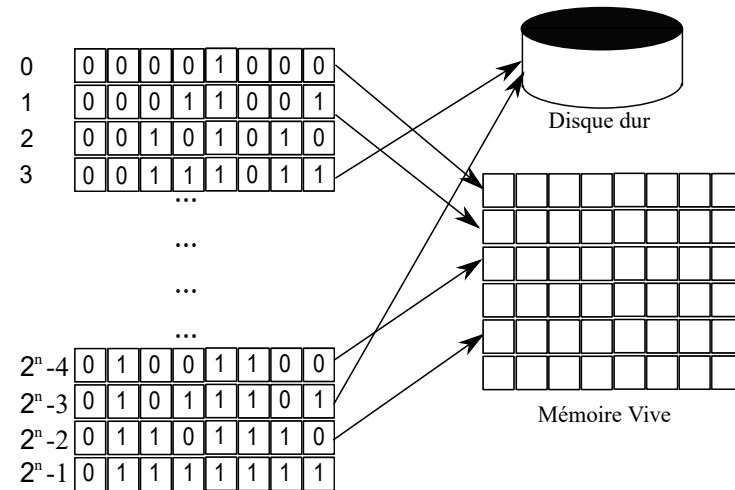
2.3. Mémoire vive

La capacité de la **mémoire vive (RAM)** d'un ordinateur est de 4 Go, 8 Go, 16 Go, etc., et elle détermine la quantité de données que l'ordinateur peut stocker temporairement pour un traitement rapide.



2.4. Mémoire Virtuelle

- La **mémoire virtuelle** étend l'utilisation de la mémoire de masse en utilisant des adresses virtuelles et physiques.
- **Adresses**: Les adresses virtuelles et les adresses physiques
 - Les adresses virtuelles sont utilisées par les programmes.
 - Les adresses physiques sont utilisées par les puces mémoire.
- **L'unité de gestion mémoire** traduit les adresses virtuelles en adresses physiques pour faciliter l'accès à la mémoire.

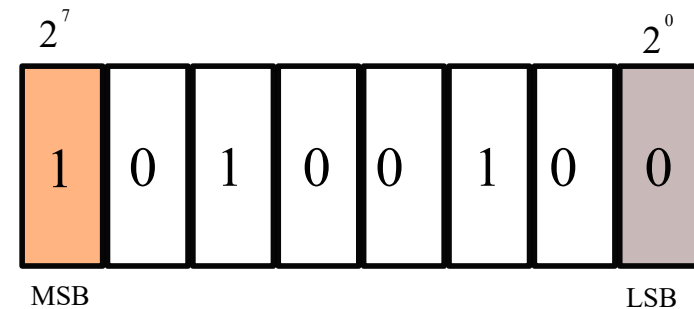


La conversion des adresses virtuelles en adresses physiques

2.5. Bit de poids fort et faible

Dans une représentation binaire donnée,

- Le **bit de poids fort** (MSB, acronyme de "Most Significant Bit") est le bit le plus à gauche et il représente la plus grande puissance de 2 dans la valeur binaire. Il a la plus grande valeur lorsque tous les autres bits sont à zéro.
- Le **bit de poids faible** (LSB, acronyme de "Least Significant Bit") est le bit le plus à droite et il représente la plus petite puissance de 2 dans la valeur binaire. Il a la plus petite valeur lorsque tous les autres bits sont à zéro.

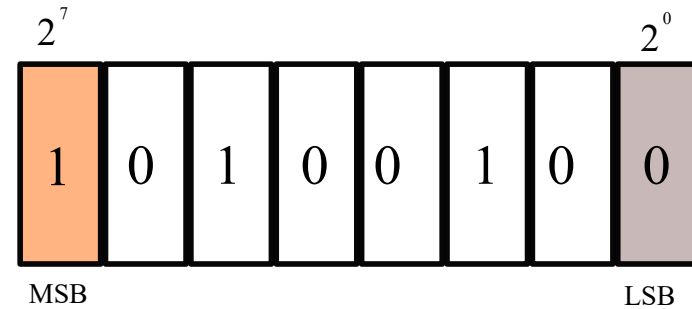


MSB et LSB (8 bit)

2.6. Endianness (Boutisme)

Dans une représentation binaire sur 8 bits (octet)

- **MSB (Bit de poids fort)** : C'est le bit le plus à gauche de l'octet, généralement le 7e bit si l'on compte de 0 à 7. Il représente la plus grande puissance de 2 dans cet octet, soit 2^7 . Sa valeur maximale est 128 lorsque ce bit est à 1.
- **LSB (Bit de poids faible)** : C'est le bit le plus à droite de l'octet, généralement le 0e bit si l'on compte de 0 à 7. Il représente la plus petite puissance de 2, soit 2^0 , et sa valeur maximale est 1 lorsque ce bit est à 1.

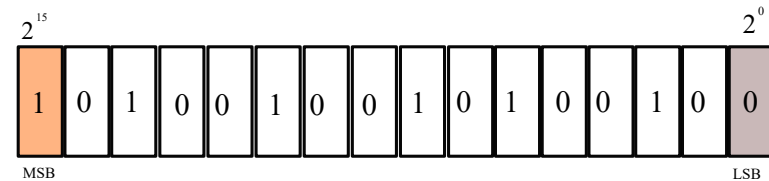


MSB et LSB (8 bit)

2.6. Endianness (Boutisme)

Dans une représentation binaire sur 16 bits

- **MSB (Bit de poids fort)** : C'est le bit le plus à gauche des 16 bits, généralement le 15e bit si l'on compte de 0 à 15. Il représente la plus grande puissance de 2 dans ces 16 bits, soit 2^{15} . Sa valeur maximale est 32 768 lorsque ce bit est à 1.
- **LSB (Bit de poids faible)**: C'est le bit le plus à droite des 16 bits, généralement le 0e bit si l'on compte de 0 à 15. Il représente la plus petite puissance de 2, soit 2^0 , et sa valeur maximale est 1 lorsque ce bit est à 1.



MSB et LSB (16 bit)

2.6. Endianness (Boutisme)

Endianness (Boutisme)

L'ordre d'organisation des octets en mémoire.

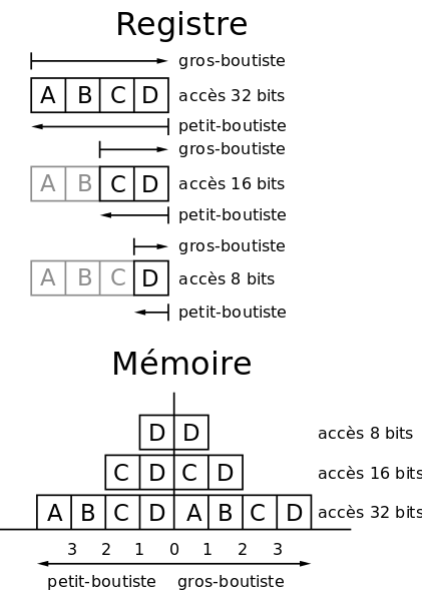
- **Big-endian (grand-boutiste) :**

Convention où l'octet le plus significatif est stocké en premier dans la mémoire.

- **Little-endian (petit-boutiste) :**

Convention où l'octet le moins significatif est stocké en premier dans la mémoire.

Remarque: Dans ce cours, nous utilisons la convention petit-boutiste.



Boutisme: grand-boutiste et petit-boutiste

2.7. Pointeurs

```
char c = 'a';
```

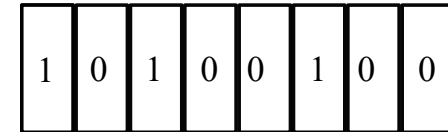
```
char *my_char_addr = &c;
```

Un pointeur est une variable qui stocke l'adresse mémoire d'une autre variable.

- Un pointeur de type `char` nommé `my_char_addr` est défini et reçoit l'adresse mémoire de la variable `c` en utilisant l'opérateur d'adresse `&`.
- `my_char_addr` fait référence à l'emplacement mémoire où la valeur `'a'` est stockée.
- **Remarque:** `my_char_addr = 0xab71`

```
char c = 0xa4;
```

&c: 0xab71



une variable char

Les liens entre les pointeurs en C, les adresses virtuelles et les adresses physiques

- **Pointeur en C** : Un pointeur en C est une variable qui stocke une adresse mémoire. Il est utilisé pour référencer et accéder à des données situées à cette adresse mémoire.
- **Adresse Physique** : L'adresse physique est l'adresse réelle à laquelle les données sont stockées en mémoire physique (RAM). Contrairement à l'adresse virtuelle, elle dépend de l'architecture matérielle de l'ordinateur.
- **Gestion de la Mémoire** : Le système d'exploitation et l'unité de gestion mémoire (MMU) sont responsables de la traduction des adresses virtuelles en adresses physiques. Cela permet au système d'exploitation de gérer la mémoire de manière efficace, notamment en utilisant des techniques telles que la pagination ou la segmentation.

Les liens entre les pointeurs en C, les adresses virtuelles et les adresses physiques

- **Protection de la Mémoire** : L'utilisation d'adresses virtuelles permet de mettre en place des mécanismes de protection de la mémoire. Par exemple, un programme ne peut pas accéder directement à la mémoire d'un autre programme en utilisant des adresses virtuelles, ce qui renforce la sécurité et l'isolation des processus.
- **Portabilité du Code** : L'utilisation d'adresses virtuelles rend le code portable entre différentes architectures matérielles, car il peut utiliser des pointeurs en C avec des adresses virtuelles sans se soucier des détails de l'adresse physique sous-jacente.

Les liens entre les pointeurs en C, les adresses virtuelles et les adresses physiques

- Les pointeurs en C utilisent des adresses virtuelles pour accéder à la mémoire, tandis que l'unité de gestion mémoire du système d'exploitation traduit ces adresses virtuelles en adresses physiques pour un accès réel à la mémoire.
- Cela permet une gestion efficace de la mémoire, la protection des processus et la portabilité du code entre différentes plateformes matérielles.

2.7. Pointeurs

`short *`

```
short s = 0xa478;  
short *my_short_addr = &s;
```

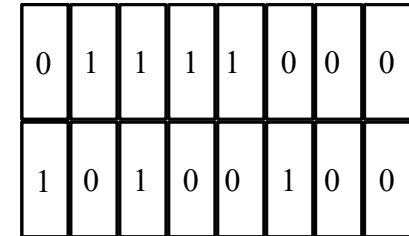
Ce code initialise une variable short `s` avec la valeur hexadécimale `0xa478`, puis crée un pointeur `my_short_addr` qui référence l'adresse mémoire de `s`.

Remarque: `my_short_addr` = `0xab71`.

`short s = 0xa478;`

`&s: 0xab71`

`0xab72`



0	1	1	1	1	0	0	0
1	0	1	0	0	1	0	0

une variable short

2.7. Pointeurs

`int *`

```
int i = 0xa47865ff;  
int *my_int_addr = &i;
```

Ce code initialise une variable `short i` avec la valeur hexadécimale `0xa47865ff`, puis crée un pointeur `my_int_addr` qui référence l'adresse mémoire de `i`.

Remarque: `my_int_addr = 0xab71`

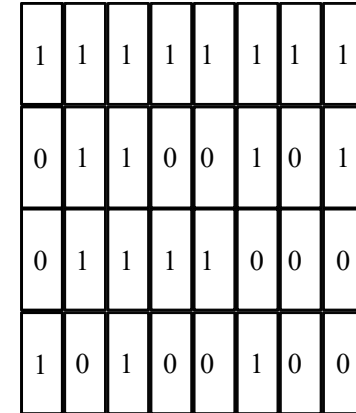
`int i = 0xa47865ff;`

`&i: 0xab71`

`0xab72`

`0xab73`

`0xab74`



1	1	1	1	1	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	1	0	0	0
1	0	1	0	0	1	0	0

une variable int

2.7. Pointeurs

long int *

```
long int li = 0xa47865ff;  
long int *my_long_int_addr = &li;
```

Ce code initialise une variable short `li` avec la valeur hexadécimale `0xa47865ff`, puis crée un pointeur `my_long_int_addr` qui référence l'adresse mémoire de `li`.

Remarque: `my_long_int_addr = 0xab71`

long int li = 0xa47865ff;

&li: 0xab71

0xab72

0xab73

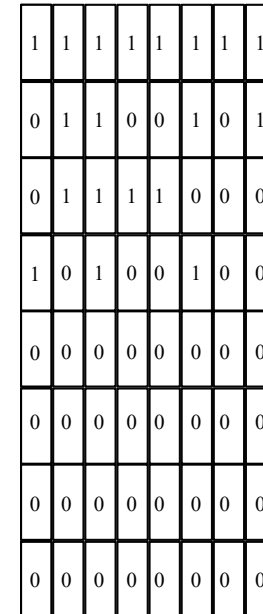
0xab74

0xab75

0xab76

0xab77

0xab78



1	1	1	1	1	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	1	0	0	0
1	0	1	0	0	1	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

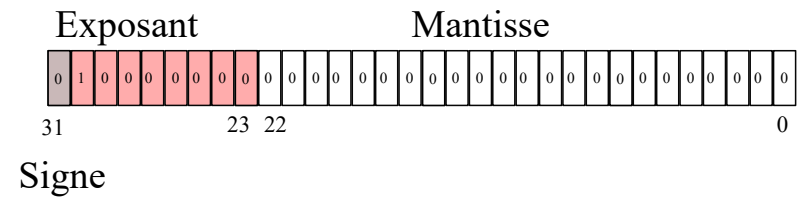
une variable long int

2.7. Pointeurs: les nombres en flottant

IEEE 754

float (32-bit)

- signe: bit 31
- exposant: bits 23-30
- mantisse: bits 0-22



une variable float

2.7. Pointeurs: les nombres en flottant

IEEE 754

`float` (32-bit): Pour stocker un numéro en utilisant la conversion IEEE 754, voici les étapes :

- **Déterminez le signe** : Si le numéro est positif, le bit de signe est 0 ; s'il est négatif, le bit de signe est 1.
- **Exprimez le numéro en binaire normalisé** : Convertissez le numéro en une forme binaire normalisée, c'est-à-dire sous la forme "1.xxxx" où "xxxx" représente la partie fractionnaire en binaire. Par exemple, pour le nombre 9.5 (1001.1), la forme normalisée en binaire est "1.0011".
- **Déterminez l'exposant** : L'exposant est le décalage nécessaire pour normaliser le numéro. Par exemple, pour "1001.1", l'exposant est 3 car il faut décaler la virgule de trois positions vers la gauche pour obtenir "1.0011".
- **Appliquez le biais** : Ajoutez le biais à l'exposant pour éviter les valeurs négatives. Pour les float 32 bits, le biais est 127. Donc, si l'exposant est 3, l'exposant stocké est $3 + 127 = 130$ en binaire.
- **Stockez le résultat** : Combinez le signe, l'exposant et la mantisse (la partie fractionnaire) pour obtenir la représentation binaire complète du numéro en virgule flottante.

2.7. Pointeurs: les nombres en flottant

IEEE 754

`float` (32-bit): Pour stocker un numéro en utilisant la conversion IEEE 754, voici les étapes pour stocker le nombre 9.5:

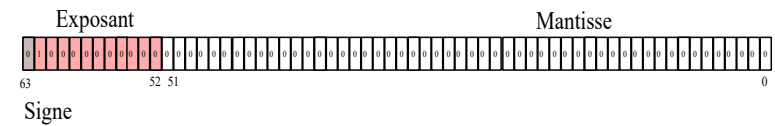
- Le signe est 0 (positif).
- La forme normalisée en binaire est "1001.1".
- L'exposant est 3.
- En ajoutant le biais (127), l'exposant stocké est 130 en binaire, soit "10000010".
- La mantisse est "001100000000000000000000" (les bits fractionnaires après le point).

2.7. Pointeurs: les nombres en flottant

IEEE 754

double (64-bit)

- signe: bit 63
- exposant: bits 52-62
- mantisse: bits 0-51



Remarque: IEEE 754

une variable double

2.7. Pointeurs: les nombres en flottant

IEEE 754

double (64-bit): La conversion d'un nombre en double précision (double) se fait généralement en plusieurs étapes :

- **Normalisation** : Le nombre est converti en une forme binaire normalisée de la forme "1.xxxx" (partie fractionnaire après la virgule) multipliée par une puissance de 2.
- **Calcul de l'exposant** : L'exposant est déterminé en fonction du décalage nécessaire pour normaliser le nombre. Dans l'exemple ci-dessus, l'exposant serait 2.
- **Application du biais** : Un décalage, appelé "biais", est ajouté à l'exposant pour le rendre positif. Le biais est de 1023.
- **Représentation des signes, exposants et mantisses** : Le bit le plus significatif est utilisé pour le signe (0 pour positif, 1 pour négatif), suivie de la représentation de l'exposant (sur un certain nombre de bits) et de la mantisse (fraction binaire après le point, sur un certain nombre de bits).
- **Assemblage des bits** : Les bits du signe, de l'exposant et de la mantisse sont combinés pour former la représentation binaire du nombre en double précision.

2.7. Pointeurs

float *

```
float f = 2;
```

```
float *my_float_addr = &f;
```

Remarque: my_float_addr = 0xab71

float f = 2;

&f: 0xab71

0xab72

0xab73

0xab74

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0

une variable float

2.7. Pointeurs

float *

```
float f = 1;
```

```
float *my_float_addr = &f;
```

Remarque: my_float_addr = 0xab71

float f = 1;

&f: 0xab71

0xab72

0xab73

0xab74

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	1	1	1	1	1	1

une variable float

2.7. Pointeurs

double *

```
double d = 2;
```

```
double *my_double_addr = &d;
```

Remarque: my_double_addr = 0xab71

double d = 2;

&d: 0xab71

0xab72

0xab73

0xab74

0xab75

0xab76

0xab77

0xab78

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0

une variable double

2.7. Pointeurs

double *

```
double d = 2;
```

```
double *my_double_addr = &d;
```

Remarque: my_double_addr = 0xab71

double d = 1;

&d: 0xab71

0xab72

0xab73

0xab74

0xab75

0xab76

0xab77

0xab78

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0
0	0	1	1	1	1	1	1

une variable double

2.7. Pointeurs

printf: Afficher à l'écran

Type	Code de conversion
pointeur	%p

```
printf("char : %p\n", my_char_addr);  
printf("short : %p\n", my_short_addr);  
printf("int : %p\n", my_int_addr);  
printf("long int : %p\n", my_long_int_addr);  
printf("float : %p\n", my_float_addr);  
printf("double : %p\n", my_double_addr);
```

2.8. L'opérateur de déréférenciation

L'opérateur de déréférenciation `*` est utilisé pour accéder à la valeur stockée à l'adresse mémoire pointée par un pointeur.

```
char c = 'a';  
char *my_char_addr = &c;  
printf ("%c", *my_char_addr);
```

- `char *my_char_addr = &c` : Un pointeur `my_char_addr` de type `char` est déclaré et reçoit l'adresse mémoire de la variable `c` à l'aide de l'opérateur d'adresse `&`. À présent, `my_char_addr` pointe vers l'emplacement en mémoire où la valeur `'a'` est stockée.
- `printf ("%c", *my_char_addr)` : L'opérateur de déréférencement (`*`) est utilisé avec `*my_char_addr` pour accéder à la valeur stockée à l'adresse mémoire pointée par `my_char_addr`, qui est `'a'`. La fonction `printf` affiche ensuite cette valeur `'a'` à l'écran.

L'opérateur de déréférencement (`*`) est utilisé pour accéder et afficher la valeur de la variable `c` à travers le pointeur `my_char_addr`.

2.8. L'opérateur de déréférenciation

Pour distinguer l'opérateur * en tant qu'opérateur de déréférencement, vous pouvez vous appuyer sur le contexte dans lequel il est utilisé.

Déclaration de pointeur : Lorsque vous déclarez un pointeur, l'opérateur * est utilisé pour indiquer qu'il s'agit d'un pointeur. Par exemple :

```
char *my_char_addr = &c;
```

Dans ce contexte, * n'est pas un opérateur de déréférencement, mais plutôt une partie de la déclaration du pointeur.

Utilisation du pointeur : Lorsque vous utilisez un pointeur pour accéder à la valeur pointée, l'opérateur * est utilisé comme opérateur de déréférencement. Par exemple :

```
printf ("%c", *my_char_addr);  
char d = *my_char_addr;
```

2.8. L'opérateur de dérérérenciation

L'opérateur * est utilisé pour manipuler un objet pointé lorsqu'il est utilisé avec un pointeur.

Exemple 1

```
char c = 'a';  
char *my_char_addr = &c;  
c = 'b';  
*my_char_addr = 'c';  
printf ("%c", c); //le caractère pointé par my_char_addr prend la valeur 'c'
```

L'opérateur * est utilisé avec `*my_char_addr` pour accéder et modifier la valeur stockée à l'adresse mémoire pointée par `my_char_addr`. Cela permet de changer la valeur de la variable `c` en modifiant directement la mémoire à l'emplacement indiqué par le pointeur.

2.8. L'opérateur de dérérérenciation

L'opérateur * est utilisé pour manipuler un objet pointé lorsqu'il est utilisé avec un pointeur.

Exemple 2

```
int i = 0x20;  
int *my_int_addr = &i;  
*my_int_addr = 1;  
*my_int_addr = *my_int_addr + 1; //i = 0x2  
i = i + 3; //i = 0x5  
printf ("%x", *my_int_addr); // 5
```

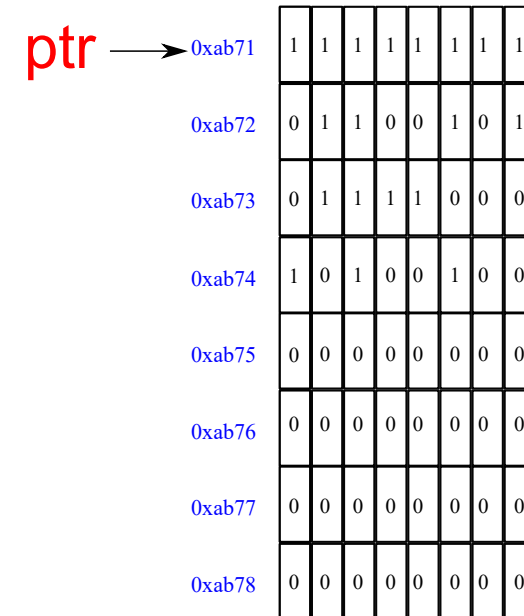
L'opérateur * est utilisé avec `*my_int_addr` pour accéder et modifier la valeur stockée à l'adresse mémoire pointée par `my_int_addr`. Cela permet de changer la valeur de la variable `i` en modifiant directement la mémoire à l'emplacement indiqué par le pointeur.

2.9. Les pointeurs et les tableaux

Pointeurs pour des blocs continus

Un **tableau** est un ensemble d'éléments homogènes.

```
char a[8] = {0xff,0x65,0x78,  
             0xa4,0x0,0x0,0x0,0x0};  
char *ptr = &a[0];
```



un tableau de caractères

2.9. Les pointeurs et les tableaux

L'opérateur d'addition sur les pointeurs est utilisé pour déplacer un pointeur vers une autre adresse mémoire en fonction du type de données auquel il pointe.

```
char a[8] = {0xff,0x65,0x78,  
             0xa4,0x0,0x0,0x0,0x0};  
char *ptr = &a[0];  
ptr = ptr + 1;
```

ptr →

0xab71	1	1	1	1	1	1	1
0xab72	0	1	1	0	0	1	0
0xab73	0	1	1	1	1	0	0
0xab74	1	0	1	0	0	1	0
0xab75	0	0	0	0	0	0	0
0xab76	0	0	0	0	0	0	0
0xab77	0	0	0	0	0	0	0
0xab78	0	0	0	0	0	0	0

un tableau de caractères

2.9. Les pointeurs et les tableaux

```
char a[8] = {0xff,0x65,0x78,  
0xa4,0x0,0x0,0x0,0x0};  
char *ptr = &a[0];  
ptr = ptr + 1;
```

- Le pointeur `ptr` pointe initialement vers le premier élément de `a`
- L'opérateur d'addition '+' est utilisé pour déplacer `ptr` d'une position vers l'avant
- Après cette opération, `ptr` pointe maintenant vers le deuxième élément de `a`

`ptr` →

0xab71	1	1	1	1	1	1	1
0xab72	0	1	1	0	0	1	0
0xab73	0	1	1	1	1	0	0
0xab74	1	0	1	0	0	1	0
0xab75	0	0	0	0	0	0	0
0xab76	0	0	0	0	0	0	0
0xab77	0	0	0	0	0	0	0
0xab78	0	0	0	0	0	0	0

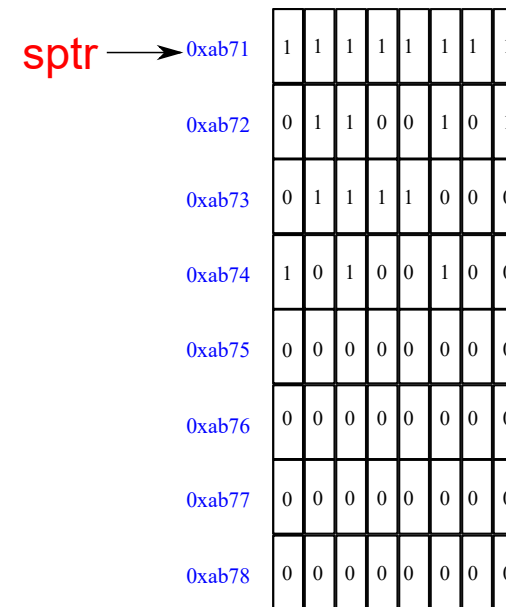
un tableau de caractères

2.9. Les pointeurs et les tableaux

L'opérateur d'addition sur les pointeurs est utilisé pour déplacer un pointeur vers une autre adresse mémoire en fonction du type de données auquel il pointe.

```
short s[4] = {0x65ff, 0xa478};  
short *sptr = &s[0];
```

Remarque: Observez bien la position de `sptr`.



un tableau d'entier

2.9. Les pointeurs et les tableaux

Remarque: Observez bien la position de `sptr`.

```
short s[4] = {0x65ff, 0xa478};
```

```
short *sptr = &s[0];
```

```
sptr = sptr + 1;
```

- Le pointeur `sptr` pointe initialement vers le premier élément de `s`
- L'opérateur d'addition '+' est utilisé pour déplacer `sptr` d'une position vers l'avant
- Après cette opération, `sptr` pointe maintenant vers le deuxième élément de `s`

`sptr` →

0xab71	1	1	1	1	1	1	1
0xab72	0	1	1	0	0	1	0
0xab73	0	1	1	1	1	0	0
0xab74	1	0	1	0	0	1	0
0xab75	0	0	0	0	0	0	0
0xab76	0	0	0	0	0	0	0
0xab77	0	0	0	0	0	0	0
0xab78	0	0	0	0	0	0	0

un tableau d'entier

2. Questions

Imaginez les trois variables suivantes: `int *iptr`, `float *fptr`, `double *dptr` . Elles ont toutes la même valeur (0x4).

```
iptr = iptr + 2;
```

```
fptr = fptr + 2;
```

```
dptr = dptr + 2;
```

Donnez les valeurs de `iptr`, `fptr`, `dptr`.

2.10. Tableaux: indices

Les indices pour les pointeurs permettent d'accéder aux éléments d'un tableau à l'aide de pointeurs.

```
short s[4] = {0x65ff, 0xa478};
```

```
short *sptr = &s[0];
```

ou

```
short *sptr = s;
```

- `sptr[0]` ou `*sptr` accède au premier élément du tableau, qui est 0x65ff.
- `sptr[1]` accède au deuxième élément du tableau, qui est 0xa478.

Les indices pour les pointeurs vous permettent de parcourir et de manipuler les éléments d'un tableau en utilisant des pointeurs, ce qui est utile pour travailler avec des tableaux dynamiques ou pour itérer à travers leurs éléments.

2.10. Tableaux: indices

Les indices pour les pointeurs permettent d'accéder aux éléments d'un tableau à l'aide de pointeurs.

```
short s[4] = {0x65ff, 0xa478};
```

```
short *sptr = s;
```

```
printf( "%hd", *(sptr+3) )
```

ou

```
printf( "%hd", *(s+3) )
```

ou

```
printf( "%hd", s[3] )
```

ou

```
printf( "%hd", sptr[3] )
```

2.11. Les pointeurs: notations

Les notations indicielles et pointeurs

Ces notations permettent de manipuler les éléments d'un tableau en utilisant soit des indices (notation indicielle) soit des pointeurs (notation pointeur).

```
short s[2] = {0x65ff, 0xa478};
```

Notation indicielle	Notation pointeur
&s[0]	s
&s[i]	s+i
s[0]	*s
s[i]	*(s+i)

2.12. Les opérateurs et les pointeurs

```
short *sptr1, *sptr2;
```

Opérateur

Exemple

pointeur + entier

`sptr1 + 2`

pointeur - entier

`sptr1 - 2`

pointeur1 - pointeur2

`sptr1 - sptr2`

- **L'opérateur pointeur + entier** : Cet opérateur permet de décaler un pointeur de 'n' positions en mémoire.
- **L'opérateur pointeur - entier** : Cet opérateur fonctionne de manière similaire à l'opérateur '+' mais en décalant le pointeur vers l'arrière.
- **L'opérateur pointeur1 - pointeur2** : Cet opérateur permet de calculer la différence en positions mémoire entre deux pointeurs.

Ces opérateurs sont souvent utilisés en programmation pour la manipulation efficace de données en mémoire, notamment lors de l'accès aux éléments d'un tableau ou lors de la gestion de la mémoire dynamique.

2.13. Les pointeurs génériques

```
void *
```

```
char a = 'a';
```

```
char *cptr = &a;
```

```
int i = 1;
```

```
cptr = &i;
```

```
$ gcc ...
```

```
warning: assignment from incompatible pointer type [-Wincompatible-pointer-types]
```

```
cptr = &i;
```

2.13. Les pointeurs génériques

void *

Un **pointeur générique** est un type spécial de pointeur qui peut contenir l'adresse mémoire de n'importe quel type de données, c'est-à-dire qu'il peut pointer vers n'importe quel type de variable. Le terme "générique" signifie que le pointeur n'a pas de type de données spécifique associé.

```
char a = 'a'; char *cptr = &a;
```

```
void *vptr = &a;
```

```
int i = 1; vptr = &i;
```

```
float f = 1; vptr = &f;
```

L'avantage principal d'un pointeur générique est sa polyvalence. Vous pouvez l'utiliser pour stocker l'adresse d'une variable de n'importe quel type,

2.14. Conversion de type

Sans perte d'information

La conversion de type, également appelée "cast", consiste à changer le type d'une variable d'un type de données à un autre. Cependant, il est important de noter que certaines conversions peuvent entraîner une perte d'information, tandis que d'autres non.

- **char-> short** : Il n'y a généralement pas de perte d'information lors de la conversion de char à short, car char est généralement un type de données d'un octet et short est généralement de deux octets. La valeur char est simplement étendue à deux octets. Cependant, si la valeur char était négative, la conversion pourrait provoquer une extension signée, ce qui signifie que les bits de signe seraient étendus pour remplir la largeur du short.
- **short-> int** : la conversion de short à int ne provoque généralement pas de perte d'information. Les valeurs short sont simplement étendues pour remplir la largeur de l'int. Encore une fois, si la valeur short était négative, l'extension signée pourrait être appliquée.
- **int-> long int** : La conversion de int à long int ne provoque généralement pas de perte d'information non plus. Les valeurs int sont simplement étendues pour remplir la largeur du long int.

2.14. Conversion de type

Avec perte d'information

- `int`-> `float` : Cette conversion peut entraîner une perte d'information, car les types `int` et `float` sont représentés différemment en mémoire. Les valeurs entières sont converties en valeurs à virgule flottante, ce qui signifie qu'elles peuvent perdre de la précision. Par exemple, un grand entier pourrait être tronqué si sa valeur est trop grande pour être représentée avec précision en tant que nombre à virgule flottante.

Exemple

```
int i = 0xffff;  
float f = i;
```

Remarque: N'oubliez pas d'utiliser `sizeof`.

Les conversions explicites

Les conversions explicites, également appelées typecasts, sont des opérations dans lesquelles vous spécifiez explicitement le type de données vers lequel vous souhaitez convertir une variable.

```
short s = 0xffff;
```

```
float f = 30.999;
```

```
int i = (int) s;
```

```
int j = (int) f;
```

Remarque: Une conversion explicite (typecasting): (type)

2.14. Conversion de type

`void *`

Lorsque vous souhaitez accéder aux données pointées par un pointeur générique, vous devez le convertir vers le type approprié.

```
char a = 'a';  
char b = 'b';  
char *cptr = &a;  
void *vptr = &b;  
  
cptr = (char *)vptr;
```

Écrivez un programme en utilisant `char *` et les opérateurs de pointeurs pour voir les octets d'une variable `long int`.

Tableau

est un ensemble d'éléments homogènes

Mais comment travailler avec un ensemble d'éléments hétérogènes?

B	o	n	j	o	u	r
---	---	---	---	---	---	---

12	1	5
----	---	---

Base de données de gestion des étudiants

```
char prenom[135][30];  
char nom[135][30];  
char rue[135][30];  
char ville[135][30];  
unsigned short notes[135];
```

Question: Quels sont les problèmes?

Base de données de gestion des étudiants

- **Taille des tableaux fixe** : Les tableaux 'prenom', 'nom', 'rue' et 'ville' sont définis avec une taille fixe de 135 éléments. Cela signifie que la base de données peut contenir un maximum de 135 étudiants. Si vous avez besoin de gérer plus d'étudiants, vous devez redimensionner manuellement ces tableaux, ce qui peut être fastidieux et source d'erreurs.
- **Taille fixe des champs** : Les champs 'prenom', 'nom', 'rue' et 'ville' sont également définis avec une taille fixe de 30 caractères. Cela peut poser problème si les noms ou les adresses des étudiants dépassent cette limite, car les données seront tronquées, ce qui peut entraîner une perte d'information.

Base de données de gestion des étudiants

- **Manque de flexibilité** : Cette base de données utilise des tableaux multidimensionnels pour stocker les informations des étudiants. Cela peut rendre la gestion des données moins flexible, notamment pour l'ajout ou la suppression d'étudiants.
- **Aucune validation des données** : Cette base de données n'inclut pas de mécanisme de validation des données. Par exemple, il n'y a pas de contrôle sur la validité des valeurs des notes (elles pourraient être en dehors de la plage attendue) ni sur les chaînes de caractères (par exemple, des adresses incorrectes pourraient être saisies).

Une structure (struct) est une construction de données en programmation permettant de regrouper plusieurs variables de types différents sous un seul nom, facilitant ainsi la manipulation et la gestion de données complexes.

```
struct etudiant{  
    char prenom[30];  
    char nom[30];  
    char rue[30];  
    char ville[30];  
    short notes;  
};
```

Avantages

- **Flexibilité** : Vous pouvez gérer un nombre variable d'étudiants en ajoutant ou en supprimant des éléments dans le tableau de structures.
- **Structure de données plus appropriée** : L'utilisation de structures permet de regrouper les données d'un étudiant en un seul objet, ce qui est plus naturel et lisible.

La déclaration et l'utilisation d'une structure

Ce code crée une variable d'une structure `etudiant`, et l'initialise avec les données spécifiques à un étudiant nommé `Dupont Pierre`.

```
struct etudiant dupont = { "Dupont", "Pierre", "Boulevard du 11 novembre 1918",  
"Villeurbanne", 19};
```

La déclaration et l'utilisation d'une structure

Ce code vise à définir une variable d'une structure "color:blue", >etudiant pour stocker les informations d'un étudiant particulier nommé "Dupont Pierre", avec une adresse et une note.

```
struct etudiant dupont;  
  
strcpy(dupont.prenom, "Dupont");  
strcpy(dupont.nom, "Pierre");  
strcpy(dupont.rue, "Boulevard du 11 novembre 1918");  
strcpy(dupont.ville, "Villeurbanne");  
dupont.notes = 19;
```

Tableaux de structures

Ce code crée un tableau de structures nommé `etudiant_cpe` pouvant contenir jusqu'à 135 étudiants, puis il initialise le premier étudiant du tableau avec les données spécifiques à "Dupont Pierre" notamment son prénom, nom, adresse et note.

```
struct etudiant etudiant_cpe[135];

strcpy(etudiant_cpe[0].prenom, "Dupont");
strcpy(etudiant_cpe[0].nom, "Pierre");
strcpy(etudiant_cpe[0].rue, "Boulevard du 11 novembre 1918");
strcpy(etudiant_cpe[0].ville, "Villeurbanne");
etudiant_cpe[0].notes = 19;
```

Une structure dans une structure

Ce code définit deux structures, `adresse` et `etudiant`, où la structure `etudiant` contient une structure `adresse` comme l'un de ses membres. Cela permet de stocker les informations de l'étudiant, y compris son prénom, nom et adresse, de manière organisée et hiérarchique.

```
struct adresse{
    char rue[30];
    char ville[30];
};
struct etudiant{
    char prenom[30];
    char nom[30];
    struct adresse adresse;
    short notes;
};
```


Une structure dans une structure

Ce code définit une structure `etudiant` qui inclut une structure imbriquée `adresse`. Cela permet de regrouper les informations relatives à un étudiant, telles que son prénom, nom et notes, ainsi que ses données d'adresse, y compris la rue et la ville, de manière structurée et organisée.

```
struct etudiant{  
    char prenom[30];  
    char nom[30];  
    struct adresse{  
        char rue[30];  
        char ville[30];  
    } adresse;  
    short notes;  
};
```

Une structure dans une structure

Ce code crée un tableau de structures `etudiant` nommé `etudiant_cpe` pouvant contenir jusqu'à 135 étudiants. Il utilise la structure imbriquée `adresse` pour stocker les détails de l'adresse de l'étudiant.

```
struct etudiant etudiant_cpe[135];  
  
strcpy(etudiant_cpe[0].prenom, "Dupont");  
strcpy(etudiant_cpe[0].nom, "Pierre");  
strcpy(etudiant_cpe[0].adresse.rue, "Boulevard du 11 novembre 1918");  
strcpy(etudiant_cpe[0].adresse.ville, "Villeurbanne");  
etudiant_cpe[0].notes = 19;
```

Créer (et Renommer) un type

Le mot-clé `typedef` est utilisé pour créer et renommer un nouveau type de données, en l'occurrence ici, un type de données structuré appelé `etudiant` qui contient des informations sur un étudiant, y compris son prénom, nom, adresse et notes.

```
typedef struct etudiant{  
    char prenom[30];  
    char nom[30];  
    struct adresse{  
        char rue[30];  
        char ville[30];  
    } adresse;  
    short notes;  
} etudiant;
```

Renommer un type

Le mot-clé `typedef` peut être utilisé pour renommer des types de données de base, tels que les types intégrés comme `int`, `char`, `float`, etc.

```
typedef int entier;
```

Les déclarations

`entier` est un alias pour le type de données `int`, ce qui signifie que vous pouvez utiliser `integer` pour déclarer des variables de type entier.

```
entier i = 10;
```

`etudiant` est un alias pour le type de données `struct etudiant`. La déclaration `etudiant dupont` crée une instance.

```
etudiant dupont;
```

2.17. Fonctions

```
/* Fichier: bonjour2.c
 * affiche un message à l'écran en utilisant un variable
 * auteur: John Samuel
 * Ceci est un commentaire sur plusieurs lignes
 */

#include <stdio.h> // en-têtes(headers)

int main() {
    int year = 2025; //déclaration d'un variable
    printf("Bonjour le Monde!!! C'est l'annee %d", year);
    return 0;
}
```

2.17. Fonctions

Les fonctions sont utilisées pour encapsuler des morceaux de code spécifiques dans un programme afin d'effectuer des opérations ou de calculer des résultats de manière modulaire et réutilisable.

Prototype: Le but de ce prototype est de fournir aux autres parties du programme des informations sur la manière d'appeler la fonction "add" et sur les types de données qu'elle attend et renvoie.

```
int add( int, int );
```

L'implémentation: Plutôt que de répéter le même code d'addition partout où vous en avez besoin dans votre programme, vous pouvez simplement appeler la fonction "add".

```
int add( int a, int b ) {  
    return a + b;  
}
```

2.17. Fonctions

Les fichiers .h servent à déclarer les prototypes de fonctions et les structures de données pour permettre une séparation claire entre l'interface publique et la définition de fonctions dans les fichiers .c.

Prototype (operators.h)

```
int add( int, int);
```

L'implémentation (operators.c)

```
int add( int a, int b ) {  
    return a + b;  
}
```

2.17. Fonctions

La déclaration d'une fonction consiste en un prototype qui spécifie le type de retour et les types des paramètres entre parenthèses, tandis que l'implémentation de la fonction utilise la même signature avec des noms de variables pour les paramètres et inclut le code à exécuter.

Prototype

```
type fonction( [type,*]);
```

L'implémentation

```
type fonction( [type variable,*]) {  
    [return valeur];  
}
```

Remarque: `type`: void, les types de base, les types composés

2.17. Fonctions

Les fichiers .h sont inclus à l'aide de `#include` pour permettre l'accès aux déclarations de fonctions, structures et variables définies dans le fichier d'en-tête, ce qui permet d'utiliser ces fonctionnalités dans d'autres parties du programme.

L'utilisation

```
#include "operators.h" // en-têtes(headers)
```

```
int num1 = 20, num2 = 60;  
int sum = add( num1, num2);
```

Remarque: Regardez l'utilisation de " " dans en-têtes

2.17. Fonctions

Prototype (nom.h)

```
void print( char *, int);
```

L'implémentation (nom.c)

```
#include <stdio.h> // en-têtes(headers)
```

```
void print( char * nom, int size ) {  
    int i;  
    for( i = 0; i < size && nom[i] != '\0'; i++) {  
        printf( "%c", nom[i]);  
    }  
}
```

Remarque: Il n'y a pas de `return`

2.17. Fonctions

```
/* Fichier: bonjour3.c
 * affiche un message à l'écran en utilisant print
 * auteur: John Samuel
 */
```

```
#include "nom.h" // en-têtes(headers)
```

```
int main() {
    print( "Bonjour le Monde!!!", 19);
    char message[] = "Pierre";
    print("Je suis ", 8);
    print(message, 19);
    return 0;
}
```

La compilation

```
$ gcc -o bonjour bonjour3.c nom.c
```

L'exécution

```
$ ./bonjour
```

```
Bonjour le Monde!!! Je suis Pierre
```

Remarque: un seul fichier C peut avoir une fonction nommée 'main'.

2.18. L'interface en ligne de commande

Les outils Linux

L'interface en ligne de commande (CLI) est un moyen d'interagir avec un système d'exploitation ou un logiciel en utilisant du texte, généralement en saisissant des commandes dans un terminal. Les **commandes** sont des instructions données à un système via une interface en ligne de commande, et les **arguments** sont des informations spécifiques fournies avec ces commandes pour indiquer ce qu'elles doivent faire

- \$ ls -l
 - \$ cd dossier
 - \$ cat fichier
-
- ls -l utilise la commande "ls" pour lister les fichiers avec l'argument "-l" qui spécifie une liste détaillée
 - cd dossier utilise la commande "cd" pour changer de répertoire vers "dossier"
 - cat fichier utilise la commande "cat" pour afficher le contenu du fichier "fichier."

2.18. L'interface en ligne de commande

```
/* Fichier: bonjour4.c
 * affiche un message à l'écran en utilisant les arguments de la ligne de commandes.
 * auteur: John Samuel
 */

#include "nom.h" // en-têtes(headers)

int main(int argc, char ** argv) {
    print("Bonjour le Monde. Je suis ", 26);
    print(argv[1], 20);
    print(argv[2], 20);
    print(argv[3], 20);
    print(argv[4], 20);
    return 0;
}
```

2.18. L'interface en ligne de commande

La compilation

```
$ gcc -o bonjour bonjour4.c nom.c
```

L'exécution

```
$/bonjour Pierre Dupont Lyon 69001
```

```
Bonjour le Monde. Je suis Pierre Dupont Lyon 69001
```

La fonction `main` prend deux arguments, `int argc` et `char ** argv`, qui permettent de passer des arguments lors de l'exécution du programme.

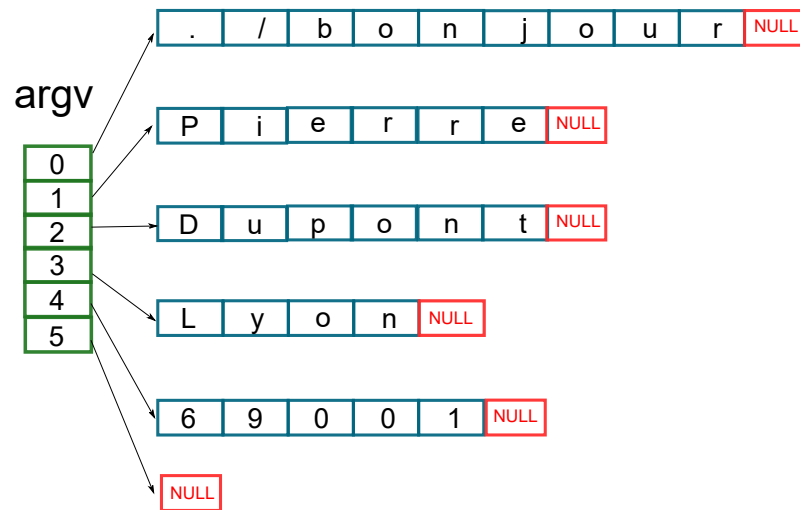
Le programme peut ainsi accéder et utiliser ces arguments pour effectuer des actions spécifiques en fonction de ces valeurs. Dans l'exemple donné, le programme `"bonjour4.c"` semble utiliser les arguments pour afficher un message de salutation personnalisé en fonction des noms et de l'adresse passés en arguments lors de son exécution.

2.18. L'interface en ligne de commande

- `int argc` représente le nombre total d'arguments passés au programme, y compris le nom du programme lui-même. Dans cet exemple, `argc` serait égal à 6, car il y a six arguments au total.
- `char ** argv` est un tableau de pointeurs vers des chaînes de caractères (tableau de chaînes de caractères) où chaque élément pointe vers un argument passé au programme. Dans cet exemple, `argv` contiendrait les éléments suivants :
 - `argv[0]` : Pointe vers le nom du programme, qui serait "bonjour" dans ce cas.
 - `argv[1]` : Pointe vers "Pierre"
 - `argv[2]` : Pointe vers "Dupont"
 - `argv[3]` : Pointe vers "Lyon"
 - `argv[4]` : Pointe vers "69001"
 - `argv[5]` : Pointe vers NULL (marquant la fin de la liste des arguments)

2.18. L'interface en ligne de commande

La compilation



2.18. L'interface en ligne de commande

```
/* Fichier: bonjour4.c
 * affiche un message à l'écran en utilisant
 * les arguments de la ligne de commandes. */

#include "nom.h" // en-têtes(headers)

int main(int argc, char ** argv) {
    print("Bonjour le Monde. Je suis ", 26);
    if ( argc == 2 ) {
        print(argv[1], 20);
    }
    return 0;
}
```

Remarque: `argv[0]` est toujours le nom de la fichier exécutable (e.g., bonjour)

2.19. La saisie d'entiers par l'utilisateur

```
/* Fichier: addition.c
 * affichage de la somme des nombres
 * saisis par l'utilisateur */
#include <stdio.h> // en-têtes(headers)

int main(int argc, char ** argv) {
    int num1, num2;
    printf("Tapez numéro 1");
    scanf("%d", &num1);
    printf("Tapez numéro 2");
    scanf("%d", &num2);
    printf("La somme: %d\n", num1 + num2);
    return 0;
}
```

L'objectif de la fonction `scanf` est de permettre à un programme de lire et de stocker des données à partir de l'entrée standard (généralement le clavier) ou d'autres flux d'entrée, en fonction d'un format spécifié.

Remarque: Regardez l'opérateur `&`.

2.19. La saisie d'entiers par l'utilisateur

```
/* Fichier: addition.c
 * affiche la somme de deux numéros
 * saisis par l'utilisateur
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)

int main(int argc, char ** argv) {
    int num1, num2;
    printf("Tapez deux numéros: ");
    scanf("%d %d", &num1, &num2);
    printf("La somme: %d\n", num1 + num2);
    return 0;
}
```

Remarque: Regardez l'opérateur `&`.
L'opérateur `&` est utilisé pour obtenir l'adresse mémoire des variables `num1` et `num2`, de sorte que les valeurs entrées par l'utilisateur puissent être stockées à ces emplacements en mémoire.

2.20. La saisie d'un mot par l'utilisateur

```
/* Fichier: bonjour5.c
 * affiche un message à l'écran en utilisant scanf.
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)

int main(int argc, char ** argv) {
    char nom[50];
    printf("Bonjour. Votre nom? ");
    scanf("%s", nom);
    printf("Bonjour %s!", nom);
    return 0;
}
```

Remarque: Regardez `nom` sans l'opérateur `&`. Lorsque vous utilisez la fonction `scanf` pour lire une chaîne de caractères à l'aide du format `"%s"`, vous n'avez pas besoin d'inclure le symbole `&` devant la variable `nom`. La raison en est que `nom` est déjà un pointeur vers un tableau de caractères (c'est-à-dire un pointeur vers la première case mémoire du tableau), et `scanf` s'attend à recevoir l'adresse de la mémoire où stocker la chaîne de caractères. Par conséquent, `nom` en tant que pointeur fournit cette adresse.

2.21. La saisie par l'utilisateur: scanf

Mots clés	Code de conversion
char	c
unsigned char	hu
short	hd
unsigned short	hu
int	d, i
unsigned int	u
long int	ld
unsigned long int	lu

2.21. La saisie par l'utilisateur: scanf

Mots clés	Code de conversion
long long int	lld
unsigned long long int	llu
float	f, F
double	g, G
long double	Lg
string of characters	s

2.22. fgets: la saisie d'une phrase par l'utilisateur

```
/* Fichier: fgets-message.c
 * affichage d'une phrase entrée par un utilisateur
 * la phrase peut contenir un espace
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)

int main(int argc, char ** argv) {
    char message[50];
    printf("Tapez votre message: ");
    fgets(message, sizeof(message), stdin);
    printf("Votre message: %s\n", message);
    return 0;
}
```

Remarque: Regardez le paramètre `stdin`. La fonction `fgets` est utilisée pour lire une ligne de texte depuis l'entrée standard (`stdin`) et stocker cette ligne dans la variable `message`. La taille du tampon de destination (`message`) est spécifiée par `sizeof(message)`, ce qui permet de garantir que la fonction ne lira pas plus de caractères que ce que le tampon peut contenir, évitant ainsi les dépassements de mémoire.

2.23. Les manuels Linux

Les manuels Linux, accessibles via la commande `man`, fournissent une documentation détaillée sur les commandes, les fonctions système et les fonctions de bibliothèque disponibles dans Linux. Par exemple, `man ls` affiche la documentation de la commande `ls`, `man 3 scanf` présente des informations sur la fonction `scanf` de la bibliothèque C.

```
$ man ls
$ man 3 scanf
$ man 3 printf
$ man 2 open
$ man fopen
..
```

Les chiffres suivant `man` indiquent la section du manuel Linux où vous trouverez des informations spécifiques : la section 1 concerne les commandes, la section 2 concerne les appels système, la section 3 concerne les appels de bibliothèque C, et ainsi de suite. Cela permet de rechercher efficacement des informations sur des sujets particuliers dans la documentation Linux.

Le manuel commence par la section **NAME**, qui indique le nom de la fonction ou de la commande couverte. La section **SYNOPSIS** fournit une syntaxe de base pour l'utilisation (de `scanf`). La section **DESCRIPTION** explique en détail comment la fonction est utilisée et à quoi elle sert. Enfin, la section **RETURN VALUE** décrit les valeurs de retour possibles de la fonction. Ces sections sont courantes dans les manuels Linux et aident les utilisateurs à comprendre et à utiliser efficacement les commandes et les fonctions.

NAME/NOM

`scanf`

...

SYNOPSIS

...

DESCRIPTION

...

RETURN VALUE/VALEUR DE RETOUR/RETOURNE

...

..

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * manipulation d'une chaîne de caractères.
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)
#include <string.h>

int main(int argc, char ** argv) {
    char nom[10];
    printf("Bonjour. Votre nom? ");
    scanf("%s", nom);
    printf("La taille: %lu\n", strlen(nom));
    return 0;
}
```

Objectif : L'objectif de la fonction `strlen` est de calculer la longueur (le nombre de caractères) d'une chaîne de caractères, c'est-à-dire le nombre de caractères qu'elle contient avant d'atteindre le caractère nul `'\0'` qui marque la fin de la chaîne.

2.24. La manipulation d'une chaîne de caractères

La compilation

```
$ gcc -o strlen string.c
```

1^{ère} Exécution

```
$/strlen
```

```
Bonjour. Votre nom? John
```

```
La taille: 4
```

2.24. La manipulation d'une chaîne de caractères

La compilation

2^{ème} Exécution

Lorsqu'un nom excessivement long est saisi, ce programme provoque une erreur de dépassement de pile (stack smashing).

```
$/strlen
```

```
Bonjour. Votre nom? : ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

```
*** stack smashing detected ***: ./strlen terminated
```

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * manipulation d'une chaîne de caractères.
 * auteur: John Samuel
 */
#include <stdio.h> // en-têtes(headers)
#include <string.h>
int main(int argc, char ** argv) {
    char nom[10];
    printf("Bonjour. Votre nom? ");
    scanf("%s", nom);
    printf("La taille: %lu\n", strlen(nom, sizeof(nom)));
    return 0;
}
```

Objectif : La fonction `strlen` a pour objectif de calculer la longueur d'une chaîne de caractères en comptant le nombre de caractères présents jusqu'à ce qu'elle atteigne une taille maximale spécifiée. Elle permet ainsi de déterminer la longueur d'une chaîne tout en évitant les dépassements de mémoire potentiels. Cela aide à garantir la sécurité et la stabilité du programme en s'assurant qu'il n'essaie pas de lire au-delà de la taille maximale autorisée pour la chaîne de caractères.

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * scanf("%[0-9]s",...).
 * auteur: John Samuel
 */
#include <stdio.h> // en-têtes(headers)
#include <string.h>
int main(int argc, char ** argv) {
    char nom[10];
    char sortie[100];
    printf("Bonjour. Votre nom? ");
    scanf("%9s", nom);
    sprintf(sortie, "La taille: %lu\n",
strlen(nom, sizeof(nom)));
    return 0;
}
```

Objectif: `scanf("%9s", nom)` lit une chaîne de caractères d'au plus 9 caractères depuis l'entrée standard et la stocke dans la variable `nom`, garantissant ainsi qu'aucun dépassement de mémoire ne se produit.

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * sprintf.
 * auteur: John Samuel
 */
#include <stdio.h> // en-têtes(headers)
#include <string.h>
int main(int argc, char ** argv) {
    char nom[10];
    char sortie[100];
    printf("Bonjour. Votre nom? ");
    scanf("%9s", nom);
    sprintf(sortie, "La taille: %lu\n", strlen(nom,
sizeof(nom)));
    return 0;
}
```

Remarque: L'objectif de ce code est de formater une chaîne de caractères contenant la taille d'une autre chaîne (**nom**) limitée à un maximum de 10 caractères, puis de stocker ce résultat dans la variable **sortie**.

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * snprintf.
 */
#include <stdio.h> // en-têtes(headers)
#include <string.h>
int main(int argc, char ** argv) {
    char nom[10];
    char sortie[100];
    printf("Bonjour. Votre nom? ");
    scanf("%9s", nom);
    snprintf(sortie, sizeof(sortie), "La taille: %lu\n",
        strlen(nom, sizeof(nom)));
    return 0;
}
```

Remarque: Utiliser `snprintf` au lieu de `sprintf` est une pratique plus sûre pour éviter les dépassements de mémoire potentiels.

2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string.c
 * atoi
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)
#include <string.h>

int main(int argc, char ** argv) {
    char numstr[10] = "1024";
    int num = atoi(numstr);
    printf("Numéro: %d", num);
    return 0;
}
```

La fonction `atoi` en C a pour objectif de convertir une chaîne de caractères représentant un nombre entier en une valeur entière de type `int`. Elle ignore les espaces initiaux, lit les chiffres jusqu'à ce qu'elle atteigne un caractère non numérique, puis retourne la valeur entière correspondante.

2.24. La manipulation d'une chaîne de caractères

```
/* sscanf */

#include <stdio.h> // en-têtes(headers)
#include <string.h>

int main(int argc, char ** argv) {
    int inum1, inum2;
    float fnum3 = 20;
    char numstr[10] = "10 20 30.33";
    sscanf(numstr, "%d %d %f", &inum1, &inum2, &fnum3);
    printf("Numéros: %d %d %f\n", inum1, inum2, fnum3);
    return 0;
}
```

La fonction `sscanf` permet de lire et de convertir des valeurs depuis une chaîne de caractères en fonction d'un format spécifié, stockant ces valeurs dans des variables, ce qui facilite l'extraction de données structurées à partir de chaînes de caractères formatées. La fonction `sscanf` analyse la chaîne `numstr` en recherchant des entiers et un nombre à virgule flottante délimités par des points-virgules, puis stocke ces valeurs dans les variables `inum1`, `fnum2`, et `fnum3`.

2.24. La manipulation d'une chaîne de caractères

```
/* sscanf */

#include <stdio.h> // en-têtes(headers)
#include <string.h>

int main(int argc, char ** argv) {
    int inum1, inum2;
    float fnum3 = 20;
    char numstr[10] = "10;20;30.33";
    sscanf(numstr, "%d;%d;%f", &inum1, &inum2, &fnum3);
    printf("Numéros: %d %d %f\n", inum1, inum2, fnum3);
    return 0;
}
```

La fonction `sscanf` analyse la chaîne `numstr` en recherchant des entiers et un nombre à virgule flottante délimités par des espaces, puis stocke ces valeurs dans les variables `inum1`, `fnum2`, et `fnum3`.

2.24. La manipulation d'une chaîne de caractères

La compilation

```
$ gcc -o strnlen string2.c
```

1^{ère} Exécution

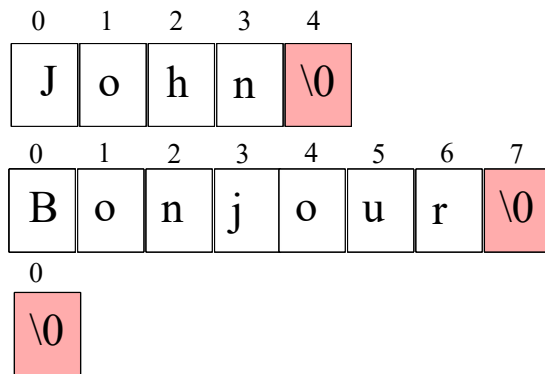
```
$ ./strnlen
```

```
Bonjour. Votre nom? John
```

```
La taille: 4
```

2.24. La manipulation d'une chaîne de caractères

- `strcat` concatene deux chaînes de caractères
- `strncat` concatene deux chaînes de caractères avec une taille maximum donnée par l'utilisateur.
- `strcpy` copie deux chaînes de caractères
- `strncpy` copie deux chaînes de caractères avec une taille maximum donnée par l'utilisateur
- `strcmp` compare deux chaînes de caractères
- `strncmp` compare deux chaînes de caractères avec une taille maximum donnée par l'utilisateur

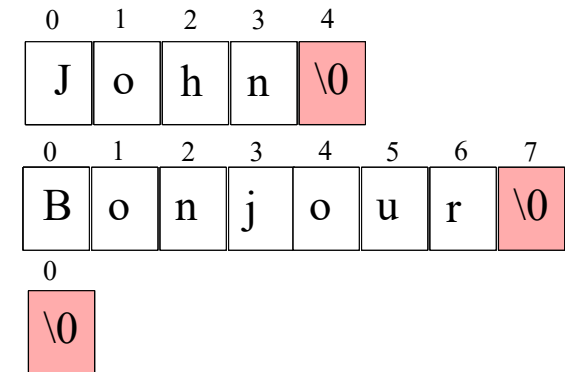


2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string3.c
 * manipulation d'une chaîne de caractères.
 */

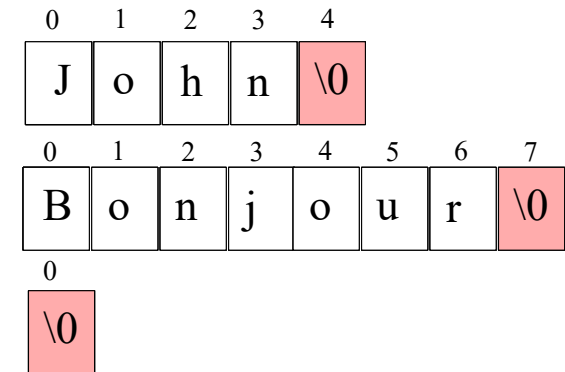
#include <stdio.h> // en-têtes(headers)
#include <string.h>

int main(int argc, char ** argv) {
    char message[] = {"Bonjour"}, nom[] = {"John"},
    string[20] = "";
    strncat(string, message, 19);
    strcat(string, " ");
    strncat(string, nom, 19);
    printf("%s", message));
    return 0;
}
```



2.24. La manipulation d'une chaîne de caractères

```
/* Fichier: string4.c
 * manipulation d'une chaîne de caractères.
 * auteur: John Samuel
 */
#include <stdio.h> // en-têtes(headers)
#include <string.h>
int main(int argc, char ** argv) {
    char message[] = {"Bonjour"}, nom[] = {"John"},
    string[20] = "";
    strncpy(string, message, 19);
    strcpy(string, " ");
    strncpy(string, nom, 19);
    printf("%s", message));
    return 0;
}
```



2.24. La manipulation d'une chaîne de caractères

La compilation

```
$ gcc -o strncat string3.c
```

```
$ gcc -o strncpy string4.c
```

1^{ère} Exécution

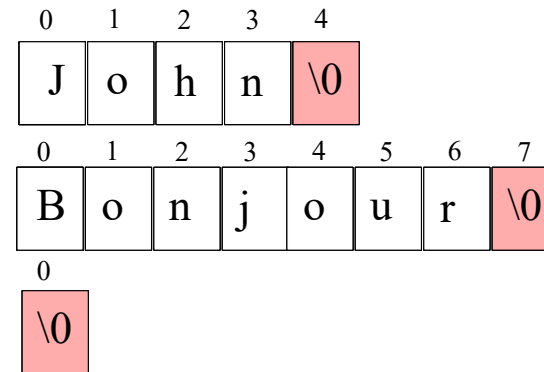
```
$ ./strncat
```

```
Bonjour John
```

2^{ème} Exécution

```
$ ./strncpy
```

```
John
```



2.25. La manipulation de fichiers

Les fonctions `open`, `read`, `write`, et `close` sont couramment utilisées pour manipuler des fichiers.

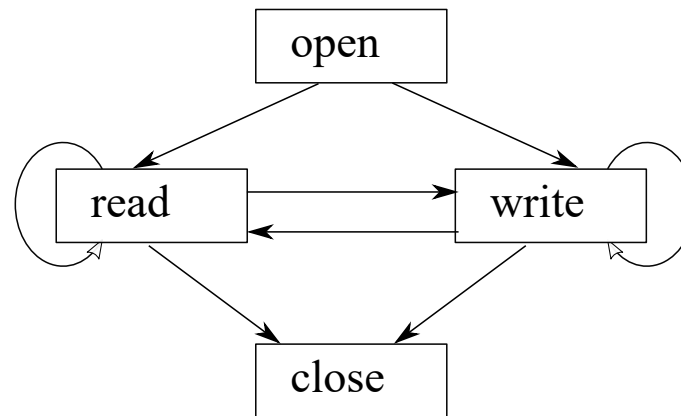
- `open` : Cette fonction sert à ouvrir un fichier existant ou à en créer un nouveau en spécifiant son chemin et son mode d'ouverture (lecture, écriture, etc.). Elle retourne un descripteur de fichier qui est utilisé pour accéder au fichier ultérieurement.
- `read` : La fonction `read` permet de lire des données depuis un fichier ouvert en utilisant le descripteur de fichier. Elle lit un nombre spécifié d'octets depuis le fichier et les stocke dans une mémoire tampon.
- `write` : Cette fonction est utilisée pour écrire des données dans un fichier ouvert. Elle prend un descripteur de fichier, une mémoire tampon contenant les données à écrire et le nombre d'octets à écrire.
- `close` : La fonction `close` permet de fermer un fichier qui a été ouvert précédemment avec `open`. Elle libère les ressources associées au descripteur de fichier et rend le fichier indisponible pour d'autres opérations.

2.25. La manipulation de fichiers

```
/* Fichier: file.c
 * Lecture d'un fichier
 */
#include <stdio.h> // en-têtes(headers)
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

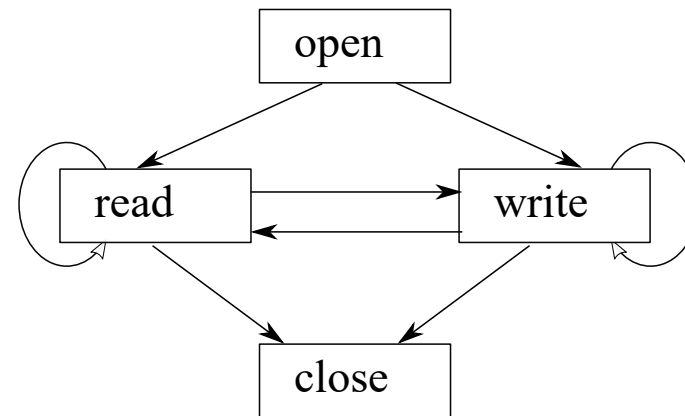
int main(int argc, char ** argv) {
    char content[1000];
    int fd, count, size;

    fd = open ("../file.c", O_RDONLY);
    size = read(fd, content, 1000);
    for (count = 0; count < size; count ++) {
        printf("%c", content[count]);
    }
    close(fd);
    return 0;
}
```



2.25. La manipulation de fichiers

```
/* Fichier: file.c
 * Lecture d'un fichier
 */
#include <stdio.h> // en-têtes(headers)
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
int main(int argc, char ** argv) {
    char content; int fd, count, size;
    fd = open ("./file.c", O_RDONLY);
    while (1) {
        size = read(fd, &content, 1);
        if (size < 1) {
            break;
        }
        printf("%c", content);
    }
    close(fd);
    return 0;
}
```



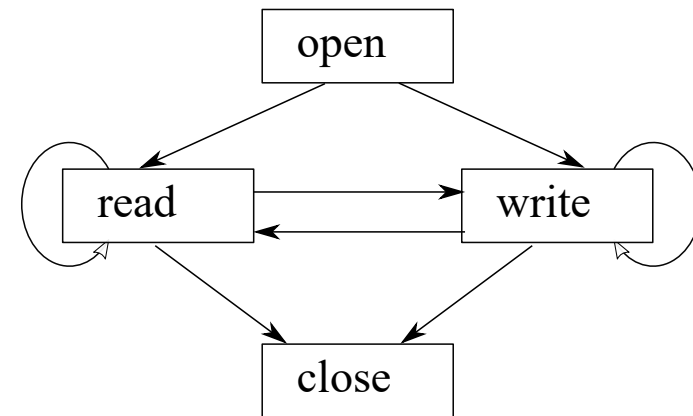
2.25. La manipulation de fichiers

```
/* Fichier: file.c
 * Écriture dans un fichier en mode ajout
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char ** argv) {
    char content[] = "Bonjour";
    int fd, count, size;

    fd = open ("../message.txt", O_CREAT|O_WRONLY, S_IRUSR|S_IWUSR);
    size = write(fd, content, sizeof(content));
    close(fd);
    return 0;
}
```



2.25. La manipulation de fichiers

```
/* Fichier: file.c
 * Écriture dans un fichier en mode ajout
 */
#include <stdio.h> // en-têtes(headers)
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char ** argv) {
    char content[] = "Bonjour";
    int fd, count, size;

    fd = open ("./message.txt", O_CREAT|O_WRONLY|O_APPEND, S_IRUSR|S_IWUSR);
    size = write(fd, content, sizeof(content));
    close(fd);
    return 0;
}
```

Lorsqu'on utilise l'option `O_APPEND` avec la fonction `open` en programmation, cela permet d'ouvrir un fichier en mode ajout, ce qui signifie que toutes les données écrites dans le fichier seront ajoutées à la fin existante plutôt que d'écraser le contenu existant.

2.26. Allocation dynamique de mémoire

```
#include <stdlib.h>
```

```
void * malloc( size_t size);
```

```
void * calloc( size_t nmemb, size_t size);
```

```
void free( void * ptr); // désallocation ou libération de mémoire
```

- **malloc** : Cette fonction alloue dynamiquement une région de mémoire de la taille spécifiée **size** en octets, et elle renvoie un pointeur vers le début de cette mémoire allouée. L'objectif est de permettre la création de structures de données dynamiques lorsqu'on ne connaît pas la taille exacte à l'avance.
- **calloc** : Cette fonction alloue dynamiquement une région de mémoire pour un tableau de **nmemb** éléments, chacun de **size** octets, **initialisée à zéro**, et elle renvoie un pointeur vers le début de cette mémoire allouée. Elle est couramment utilisée pour allouer de la mémoire pour des tableaux.
- **free** : Cette fonction libère la mémoire précédemment allouée dynamiquement à l'aide de **malloc** ou **calloc**. Son objectif est de libérer la mémoire inutilisée afin d'éviter les fuites de mémoire et d'optimiser l'utilisation des ressources mémoire dans un programme.

2.26. Allocation dynamique de mémoire

```
/* Fichier: memory.c
 * Allocation dynamique de mémoire
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)
#include <stdlib.h>

int main(int argc, char ** argv) {
    char *content = malloc(10 * sizeof(char));
    strncat(content, "Bonjour", 10);
    free(content); //libération de mémoire
    return 0;
}
```


2.26. Allocation dynamique de mémoire

```
/* Fichier: memory.c
 * Allocation dynamique de mémoire
 * auteur: John Samuel
 */

#include <stdio.h> // en-têtes(headers)
#include <stdlib.h>

int main(int argc, char ** argv) {
    char *content = calloc(10, sizeof(char));
    strncat(content, "Bonjour", 10);
    free(content); // libération de mémoire
    return 0;
}
```

Écrivez un programme qui réserve et libère un espace mémoire pour un tableau d'entiers (int, long int, short ou long long int) et un tableau de nombres en flottant (float, double ou long double) en utilisant malloc, calloc et free.

Union

Une union est déclarée comme une structure. L'objectif d'une union en programmation est de permettre le partage d'un même espace mémoire par plusieurs variables de types différents. La taille de l'union est déterminée par la taille de son élément le plus grand, ce qui permet d'économiser de la mémoire en ne réservant qu'un seul emplacement mémoire pour plusieurs utilisations potentielles.

```
union etudiant{  
    char prenom[30];  
    char nom[30];  
    char rue[30];  
    char ville[30];  
};
```

Remarque: La structure union `etudiant` peut stocker différentes informations telles que le prénom, le nom, la rue et la ville, mais elle ne peut stocker qu'un seul de ces éléments à la fois.

La déclaration et l'utilisation d'une union

```
union etudiant dupont;
```

```
strcpy(dupont.nom, "Dupont");  
printf("%s", dupont.prenom);  
printf("%s", dupont.rue);  
printf("%s", dupont.ville);
```

L'affichage

Dupont

Dupont

Dupont

Remarque: Tous les éléments partagent la même espace de mémoire.

Avantages

```
struct content{  
    char type[30];  
    union{  
        char * ccontent;  
        int * icontent;  
        float * fcontent;  
    };  
};
```

- **Économie d'espace mémoire** : Les unions permettent de stocker différents types de données dans le même espace mémoire, mais seulement un type à la fois. Cela réduit la consommation de mémoire par rapport à l'utilisation de variables distinctes pour chaque type de données.
- **Flexibilité polymorphe** : Les unions sont utiles lorsque la nature des données à stocker peut changer dynamiquement. Vous pouvez économiser de la mémoire en n'allouant de l'espace que pour le type de données actuellement utilisé.
- **Structures compactes** : Les unions sont idéales pour la création de structures compactes qui peuvent stocker une variété de types de données, ce qui est particulièrement utile dans les situations où l'espace mémoire est limité.

2.27. Les unions

```
/* Fichier: memory3.c
 * Allocation dynamique de mémoire */
#include <stdio.h> // en-têtes(headers)
#include <stdlib.h>
int main(int argc, char ** argv) {
    struct content c;
    strcpy(c.type, "char", 10);
    if (strcmp(c.type, "char") == 0) {
        c.ccontent = calloc(10, sizeof(char));
    }
    else if (strcmp(c.type, "int") == 0) {
        c.icontent = calloc(10, sizeof(int));
    }
    return 0;
}
```

2.27. Les unions

Point3D

La structure union `point3d` permet de stocker des coordonnées tridimensionnelles en utilisant soit un tableau d'entiers de taille 3 (`value[3]`) soit une structure avec des membres `x`, `y`, et `z`. Cela signifie que vous pouvez accéder aux coordonnées de deux manières différentes, en fonction de l'utilisation. Par exemple, vous pouvez utiliser `point3d.value[0]` pour accéder à la coordonnée `x` en utilisant le tableau d'entiers, ou `point3d.x` pour accéder à la coordonnée `point3d.value[0]` en utilisant la structure. Cela offre une certaine flexibilité lors de la manipulation de coordonnées tridimensionnelles, selon le contexte de votre programme.

```
union point3d{
    int value[3];
    struct{
        int x;
        int y;
        int z;
    };
};
```

2.27. Les unions

```
/* Fichier: union.c
 * la déclaration et l'utilisation d'une union
 */

#include <stdio.h> // en-têtes(headers)

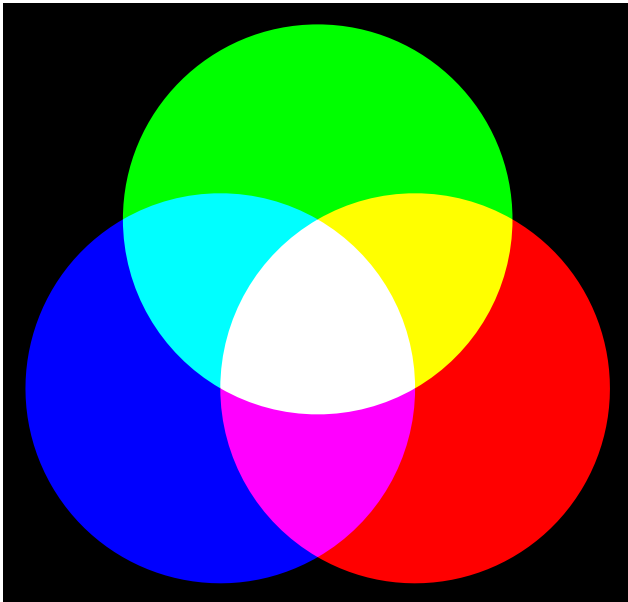

int main(int argc, char ** argv) {
    union point3d p;
    p.value[0] = 0x10;
    p.value[1] = 0x45;
    p.value[2] = 0x78;

    printf("%x %x %x\n", p.x, p.y, p.z);
    return 0;
}
```


2. Question

Écrivez une structure pour la représentation d'une couleur RGB

(rouge, vert, bleu: chaque couleur prend un octet) en utilisant union et struct.



Source: <https://commons.wikimedia.org/wiki/>

File:AdditiveColorMixing.svg

2.28. Débogage et test

Le code suivant (**erreurs.c**) présente une erreur majeure : il tente d'accéder à des indices de tableau supérieurs à la taille maximale du tableau (100).

```
#include <stdio.h>

int main() {

    int tableau[100];

    for (int compteur = 0; compteur < sizeof(tableau); compteur++) { //Erreur
        tableau[compteur] = tableau[compteur] * 2;
    }

    return (0);

}
```

2.28. Débogage et test

Pour la compilation et la création d'un exécutable

```
$ gcc erreurs.c
```

Lorsque nous exécutons le code, nous obtenons l'erreur suivante et le programme se plante (comme prévu).

```
./a.out  
*** stack smashing detected ***: terminated  
fish: Job 1, './a.out' terminated by signal SIGABRT (Abort)
```

2.28. Débogage et test

Pour déboguer ce code et trouver la source de l'erreur, nous devons d'abord le compiler et ajouter des informations supplémentaires pour le débogage avec l'option `-ggdb3`.

```
$ gcc --ggdb3 erreurs.c
```

2.28. Débogage et test

Nous allons maintenant exécuter le code avec gdb.

```
$ gdb a.out
GNU gdb (Ubuntu 12.0.90-0ubuntu1) 12.0.90
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from a.out...
(gdb)
```

2.28. Débogage et test

Une fois que vous exécutez gdb, vous verrez une autre ligne de commande avec `(gdb)`. Tapez `r` pour exécuter le programme.

```
(gdb) r
Starting program: ./ProgC/gdb/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
*** stack smashing detected ***: terminated

Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (no_tid=0, signo=6, threadid=140737351481152) at ./nptl/pthr
44      ./nptl/pthread_kill.c: No such file or directory.
```

2.28. Débogage et test

Le programme s'est écrasé et pour trouver la ligne qui est la raison de l'écrasement, tapez `bt`.

```
(gdb) bt
```

```
#0  __pthread_kill_implementation (no_tid=0, signo=6, threadid=140737351481152) at ./nptl/  
#1  __pthread_kill_internal (signo=6, threadid=140737351481152) at ./nptl/pthread_kill.c:7  
#2  __GI___pthread_kill (threadid=140737351481152, signo=signo@entry=6) at ./nptl/pthread_  
#3  0x00007ffff7dbc476 in __GI_raise (sig=sig@entry=6) at ../sysdeps/posix/raise.c:26  
#4  0x00007ffff7da27f3 in __GI_abort () at ./stdlib/abort.c:79  
#5  0x00007ffff7e036f6 in __libc_message (action=action@entry=do_abort, fmt=fmt@entry=0x7f  
    "*** %s ***: terminated\n")  
    at ../sysdeps/posix/libc_fatal.c:155  
#6  0x00007ffff7eb076a in __GI___fortify_fail (msg=msg@entry=0x7ffff7f5592b "stack smashir  
    at ./debug/fortify_fail.c:26  
#7  0x00007ffff7eb0736 in __stack_chk_fail () at ./debug/stack_chk_fail.c:24  
#8  0x00005555555551c1 in main () at erreurs.c:13  
(gdb)
```

2.28. Débogage et test

Nous voyons que l'erreur se situe à la ligne 13 du programme (erreurs.c). Vous pouvez maintenant quitter gdb en utilisant `quit`, corriger le code et répéter les étapes ci-dessus jusqu'à ce que l'erreur soit résolue.

```
(gdb) quit
```

```
A debugging session is active.
```

```
Inferior 1 [process 3886] will be killed.
```

```
Quit anyway? (y or n) y
```


Références

- Langage C, Claude Delannoy
- https://fr.wikipedia.org/wiki/Architecture_32_bits
- <https://en.wikipedia.org/wiki/X86>
- <https://fr.wikipedia.org/wiki/Endianness>
- https://fr.wikipedia.org/wiki/IEEE_754

Crédits d'images

- [Wikimedia Commons](#)